

# StudyBuddy Neural Study Engine

A production-ready neural study engine that generates flashcards, quizzes, subjective practice questions, and examiner-style feedback from any study document.

---

## Architecture

Client App

```
└─> window.StudyBuddyEngine (adapter.js)
    └─> Backend: POST /api/study-engine ──> Fine-tuned Phi-3 Mini (QLoRA)
    └─> Fallback: fallback.js (rule-based, offline)
```

**Model:** `microsoft/Phi-3-mini-4k-instruct` fine-tuned via QLoRA on educational QA datasets.

**Tasks:** Question generation, flashcard generation, quiz generation, answer evaluation.

**Method:** Single multi-task model with task-prefix prompts, trained with SFTTrainer + LoRA.

---

## Quick Start

### 1. Install dependencies

```
bash

pip install -r requirements.txt
```

### 2. Download and prepare datasets

```
bash

cd model-training
python scripts/prepare_data.py --output_dir data/processed
```

This will automatically download the following public datasets from HuggingFace:

| Dataset            | HuggingFace ID                         | Used For                            |
|--------------------|----------------------------------------|-------------------------------------|
| SQuAD v2           | <code>rajpurkar/squad_v2</code>        | Document → question generation base |
| RACE (high school) | <code>ehovy/race</code>                | MCQ generation                      |
| QuALITY            | <code>allenai/quality</code>           | Long document comprehension QA      |
| OpenBookQA         | <code>allenai/openbookqa</code>        | Concept-level question generation   |
| FairytaleQA        | <code>WorkInTheDark/FairytaleQA</code> | Document-to-question alignment      |
| ASAP AES (Kaggle)  | <i>(manual — see note below)</i>       | Answer scoring / evaluation         |
| SciQ               | <code>allenai/sciq</code>              | Science domain MCQ + explanations   |

**ASAP AES Note:** Download from <https://www.kaggle.com/c/asap-aes/data> Place the `training_set_rel3.tsv` file at `model-training/data/raw/asap_aes.tsv` before running `prepare_data.py`. This is the primary answer-evaluation corpus.

The script then:

- Normalises all formats into the unified training schema
- Splits train / validation / test **by document** (no leakage)
- Generates synthetic flashcard and quiz examples from SQuAD passages
- Applies negative examples (vague, hallucinated, copied answers)
- Saves `train.jsonl`, `val.jsonl`, `test.jsonl` under `data/processed/`

### 3. Train the model

```
bash

python scripts/train_lora.py --config configs/generation.yaml
```

Requires a GPU with  $\geq 16$  GB VRAM (24 GB recommended). On an A100-40GB, training completes in approximately 6–10 hours for the 50k example set.

For lower VRAM (8–12 GB), use the `--load_in_4bit` flag:

```
bash

python scripts/train_lora.py --config configs/generation.yaml --load_in_4bit
```

## 4. Evaluate

```
bash

python scripts/evaluate.py \
  --model_path outputs/studybuddy-phi3-lora \
  --test_file data/processed/test.jsonl
```

## 5. Export model

```
bash

python scripts/export_model.py \
  --model_path outputs/studybuddy-phi3-lora \
  --output_dir outputs/studybuddy-merged \
  --format gguf          # or: hf, onnx
```

## 6. Run the backend server

```
bash

cd study-engine/server
uvicorn server:app --host 0.0.0.0 --port 8000 --reload
```

---

## Project Structure

```
study-buddy-neural-engine/
├─ README.md
├─ requirements.txt
├─
├─ model-training/
│   └─ data/
│       ├── raw/          ← downloaded source datasets land here
│       ├── processed/    ← train.jsonl, val.jsonl, test.jsonl
│       └─ eval/          ← human-eval samples
│   └─ scripts/
│       ├── prepare_data.py ← downloads + normalises all datasets
│       ├── train_lora.py   ← QLoRA fine-tuning with TRL SFTTrainer
│       ├── evaluate.py     ← JSON validity, rubric coverage, score corr.
│       └─ export_model.py ← merge LoRA + export to HF / GGUF / ONNX
│   └─ configs/
│       └─ generation.yaml ← training hyperparameters
```

```
| | └─ evaluation.yaml ← eval thresholds and metrics config
| └─ notebooks/
|   └─ explore_data.ipynb
|
└─ study-engine/
  └─ adapter.js          ← window.StudyBuddyEngine browser global
  └─ fallback.js        ← offline rule-based fallback
  └─ server/
    └─ server.py         ← FastAPI POST /api/study-engine
    └─ engine.py         ← model loading + inference logic
    └─ prompts.py        ← all prompt templates
  └─ tests/
    └─ test_outputs.py   ← JSON schema, hallucination, scoring checks
    └─ test_integration.py ← end-to-end adapter integration tests
    └─ sample_data.json  ← small fixture for CI
```

## Target Metrics

| Metric                                           | Target  |
|--------------------------------------------------|---------|
| JSON valid rate                                  | > 98%   |
| Human-acceptable questions                       | > 85%   |
| Score correlation with human graders (Pearson r) | > 0.75  |
| Hallucinated / unsupported facts                 | < 5%    |
| p95 backend latency                              | < 5 s   |
| p95 local fallback latency                       | < 1.5 s |

## Replacement / Removal

All model logic lives in `study-engine/`. The app depends only on the adapter public API. To swap the engine, replace `adapter.js` and `server/` without touching any UI component.

```
js
```

```
// Only this contract must be preserved:
window.StudyBuddyEngine.makeQuestions({ notes, topic, count })
window.StudyBuddyEngine.evaluateAnswer({ student, expected, question, topic, rubric
window.StudyBuddyEngine.makeFlashcards({ notes, topic, count })
window.StudyBuddyEngine.makeQuiz({ notes, topic, count, difficulty })
```